

Complex Java 8 Case Study

Global Automotive Spare Parts Order and Supply Chain Management System

1. Case Study Overview

Volkswagen operates regional spare-parts distribution centres that supply components to authorised service centres and dealerships.

The company handles thousands of orders involving:

- Engine components
- Brake assemblies
- Batteries
- Electrical parts
- Filters
- Tyres
- Transmission components
- Body parts
- Safety equipment

Each service centre raises orders based on vehicle repairs and scheduled maintenance requirements.

The existing system collects order, inventory, supplier and shipment data from different regional applications. Preparing reports and identifying delayed, urgent or high-value orders requires significant manual work.

Volkswagen wants to develop a Java-based application called:

Global Automotive Spare Parts Order and Supply Chain Management System

The application will use Java 8 features to process large collections of:

- Parts
- Suppliers
- Warehouses
- Dealers
- Orders
- Order items
- Shipments
- Payments
- Inventory transactions

2. Business Problem

The current order and inventory process has the following challenges:

- Stock information is maintained separately by each warehouse.
- Service centres do not know the nearest warehouse with available stock.
- High-priority orders are not always processed first.
- Delayed shipments are identified manually.
- Supplier performance reports take several days to prepare.
- Duplicate part orders are difficult to identify.
- Inventory shortages are discovered only after orders are placed.
- Order totals and discounts are sometimes calculated incorrectly.
- Missing order, supplier or shipment data may cause application errors.
- Reports for different warehouses are generated sequentially.
- Management cannot quickly identify high-demand spare parts.
- Expired and obsolete parts remain in inventory.
- Emergency orders are not automatically prioritised.
- Backordered items are not tracked efficiently.
- Shipment and payment operations delay order confirmation.

The new system should automate these operations using Java 8 capabilities.

3. Project Objectives

The application should:

- Manage automotive spare-part records.
 - Maintain warehouse inventory.
 - Process dealer and service-centre orders.
 - Validate stock before confirming an order.
 - Calculate taxes, discounts and shipping costs.
 - Prioritise emergency and high-value orders.
 - Identify delayed and partially fulfilled orders.
 - Analyse supplier performance.
 - Generate warehouse-level reports.
 - Process multiple reports asynchronously.
 - Handle missing information safely.
 - Generate business insights using Java 8 Stream API.
 - Support layered application architecture.
-

4. Major System Modules

The application should contain the following modules:

1. Part Management
 2. Supplier Management
 3. Warehouse Management
 4. Inventory Management
 5. Dealer and Service-Centre Management
 6. Order Management
 7. Shipment Management
 8. Payment Management
 9. Supplier Performance Analysis
 10. Inventory Forecasting
 11. Reporting and Analytics
 12. Notification Management
-

5. Core Business Entities

5.1 Part

Each spare part should contain:

Field	Description
Part ID	Unique part identifier
Part name	Name of the spare part
Part category	Engine, brake, electrical, body, etc.
Vehicle model	Compatible vehicle model
Unit price	Selling price of one part
Manufacturing date	Date of manufacture
Expiry date	Expiry date, where applicable
Minimum stock level	Minimum required inventory
Reorder quantity	Quantity to order when stock is low
Weight	Weight used for shipping calculation
Critical part	Indicates whether the part is safety-critical
Active status	Active, obsolete or discontinued

5.2 Supplier

Each supplier should contain:

- Supplier ID
- Supplier name
- Supported part categories
- City
- Country
- Contact details
- Contract start date
- Contract end date
- Average delivery time
- Quality rating
- On-time delivery percentage
- Rejected shipment count
- Active status

5.3 Warehouse

Each warehouse should contain:

- Warehouse ID
- Warehouse name
- City
- Region
- Storage capacity
- Current utilisation
- Supported part categories
- Manager name
- Operational status

Example warehouses:

- Pune Distribution Centre
 - Mumbai Distribution Centre
 - Bengaluru Distribution Centre
 - Delhi Distribution Centre
 - Chennai Distribution Centre
 - Hyderabad Distribution Centre
-

5.4 Inventory Record

Each inventory record should contain:

- Inventory ID
 - Warehouse ID
 - Part ID
 - Available quantity
 - Reserved quantity
 - Damaged quantity
 - Reorder level
 - Last restock date
 - Last issue date
 - Storage location
 - Batch number
-

5.5 Dealer or Service Centre

Each service centre should contain:

- Centre ID
 - Centre name
 - Dealer type
 - City
 - Region
 - Contact person
 - Credit limit
 - Outstanding balance
 - Priority category
 - Active status
-

5.6 Customer Order

Each order should contain:

- Order ID
- Service-centre ID
- Order date
- Required delivery date
- Order priority
- Order status
- List of order items
- Assigned warehouse
- Gross amount

- Discount amount
 - Tax amount
 - Shipping amount
 - Final payable amount
 - Payment status
 - Shipment status
-

5.7 Order Item

Each order item should contain:

- Order item ID
 - Part ID
 - Requested quantity
 - Available quantity
 - Approved quantity
 - Backordered quantity
 - Unit price
 - Discount percentage
 - Item total
-

5.8 Shipment

Each shipment should contain:

- Shipment ID
 - Order ID
 - Warehouse ID
 - Logistics provider
 - Dispatch date
 - Expected delivery date
 - Actual delivery date
 - Shipment status
 - Tracking number
 - Delivery location
 - Shipping cost
-

5.9 Payment

Each payment should contain:

- Payment ID

- Order ID
 - Payment date
 - Payment method
 - Amount
 - Payment status
 - Transaction reference
 - Refund amount
-

6. Order Priority Categories

Orders should be classified as:

Priority	Description
Critical	Vehicle safety issue or vehicle off-road condition
Emergency	Part required within 24 hours
High	Important repair with short delivery expectation
Normal	Standard maintenance order
Low	Stock replenishment or non-urgent requirement

Critical and emergency orders should be processed before normal orders.

7. Order Status Workflow

An order may move through the following statuses:

1. Created
2. Validated
3. Stock Verification Pending
4. Partially Available
5. Backordered
6. Approved
7. Payment Pending
8. Payment Confirmed
9. Ready for Dispatch
10. Shipped
11. Partially Delivered
12. Delivered
13. Cancelled
14. Returned
15. Closed

The system should validate allowed transitions between these statuses.

8. Functional Requirements

8.1 Display Active Spare Parts

Display all parts whose status is active.

Exclude:

- Obsolete parts
- Discontinued parts
- Expired parts

Java 8 concepts

- Predicate
 - Lambda expression
 - Stream filtering
 - Date and Time API
-

8.2 Filter Parts by Vehicle Model

The user should provide a vehicle model and retrieve compatible spare parts.

Example models:

- Polo
- Vento
- Virtus
- Taigun
- Tiguan

Java 8 concepts

- Predicate
 - `filter()`
 - Case-insensitive comparison
-

8.3 Filter Parts by Category

Retrieve parts belonging to a selected category.

Examples:

- Engine
- Braking
- Electrical
- Suspension
- Transmission
- Safety

Java 8 concepts

- Lambda expressions
 - Stream API
 - Predicate
-

8.4 Identify Critical Parts

Generate a report of safety-critical parts such as:

- Brake pads
- Brake discs
- Airbags
- Seat-belt components
- Steering parts
- ABS sensors

Java 8 concepts

- Predicate
 - Method references
 - Collectors
-

8.5 Identify Expired Parts

Identify parts whose expiry date is earlier than the current date.

The report should include:

- Part name
- Batch number
- Warehouse
- Expiry date
- Available quantity
- Estimated inventory loss

Java 8 concepts

- `LocalDate`
 - Date comparison
 - Filtering
 - Mapping
-

8.6 Identify Parts Expiring Soon

Find parts that will expire within the next:

- 30 days
- 60 days
- 90 days

Java 8 concepts

- `LocalDate`
 - `Period`
 - Date-range filtering
-

8.7 Identify Low-Stock Parts

A part should be marked as low stock when:

Available quantity is less than or equal to the reorder level.

The report should be generated warehouse-wise.

Java 8 concepts

- Predicate
 - `filter()`
 - `groupingBy()`
-

8.8 Identify Out-of-Stock Parts

Identify inventory records where the available quantity is zero.

The system should separate:

- Critical out-of-stock parts

- Non-critical out-of-stock parts

Java 8 concepts

- Predicate composition
 - Partitioning
 - Stream API
-

8.9 Calculate Usable Stock

Usable stock should be calculated as:

```
Available quantity - reserved quantity - damaged quantity
```

The system should transform inventory records into stock-summary records.

Java 8 concepts

- Function
 - `map()`
 - Stream transformation
-

8.10 Find the Nearest Warehouse with Stock

When an order is placed, the system should:

1. Identify warehouses with sufficient usable stock.
2. Exclude non-operational warehouses.
3. Prefer a warehouse in the same region.
4. Select the warehouse with the lowest estimated delivery time.
5. Select another warehouse if the preferred warehouse cannot fulfil the complete quantity.

Java 8 concepts

- Multiple predicates
 - Sorting
 - Comparator chaining
 - Optional
 - `findFirst()`
-

8.11 Validate an Order

Before processing an order, validate:

- Service centre exists and is active.
- Order contains at least one item.
- Requested quantities are positive.
- All part IDs are valid.
- Parts are active and not expired.
- Dealer credit limit is sufficient.
- Required delivery date is valid.
- Duplicate parts are not entered unnecessarily.
- Critical orders contain a valid emergency reason.

Java 8 concepts

- Predicate
 - Predicate chaining
 - `allMatch()`
 - `anyMatch()`
 - `noneMatch()`
-

8.12 Detect Duplicate Order Items

Identify whether the same part is included multiple times in one order.

The system should either:

- Reject duplicate items, or
- Merge duplicate quantities into a single order item

Java 8 concepts

- Grouping
 - Counting
 - Stream collection
 - Set-based validation
-

8.13 Calculate Item Total

For every order item, calculate:

`Approved quantity × unit price`

Apply part-level discount where applicable.

Java 8 concepts

- Function
 - `map()`
 - Functional interface
-

8.14 Calculate Order Gross Amount

Calculate the total amount of all approved order items before tax, discount and shipping.

Java 8 concepts

- `mapToDouble()`
 - `sum()`
 - `reduce()`
-

8.15 Apply Order Discount

Discount should depend on order value and customer priority.

Discount rules

Condition	Discount
Order above ₹10,00,000	10%
Order above ₹5,00,000	7%
Order above ₹2,00,000	5%
Premium service centre	Additional 2%
Emergency order	No additional commercial discount

The maximum discount should not exceed 12%.

Java 8 concepts

- Custom functional interface
 - Lambda expression
 - Function composition
-

8.16 Calculate Tax

Tax should be calculated after discount.

Different part categories may have different tax rates.

Java 8 concepts

- Functional interface
 - Function
 - Map-based rate lookup
 - Stream aggregation
-

8.17 Calculate Shipping Cost

Shipping cost should depend on:

- Total shipment weight
- Delivery distance
- Order priority
- Logistics provider
- Delivery region

Critical and emergency orders may have premium shipping charges.

Java 8 concepts

- Custom functional interface
 - Lambda expression
 - Reduction
 - Conditional mapping
-

8.18 Calculate Final Payable Amount

The final amount should be calculated as:

Gross amount - discount + tax + shipping charges

The system should generate a complete pricing summary.

8.19 Allocate Inventory

For every approved item:

- Reduce available stock.
- Increase reserved stock.
- Record the allocated warehouse.
- Calculate backordered quantity when stock is insufficient.
- Prevent stock from becoming negative.

Java 8 concepts

- Stream processing
 - Filtering
 - Optional
 - Functional validation
-

8.20 Process Partial Order Fulfilment

When the complete quantity is unavailable:

- Approve available quantity.
- Mark remaining quantity as backordered.
- Update order status to partially available.
- Search another warehouse for remaining stock.
- Generate a partial-fulfilment report.

Java 8 concepts

- Stream pipelines
 - Grouping
 - Optional
 - Flat mapping
-

8.21 Split an Order Across Warehouses

One order may be fulfilled using inventory from multiple warehouses.

The system should:

- Group allocated items by warehouse.
- Create separate shipment requests.
- Calculate shipment totals warehouse-wise.
- Combine shipment details under the original order.

Java 8 concepts

- `groupingBy()`
 - `flatMap()`
 - Collectors
 - Map processing
-

8.22 Prioritise Orders

Sort pending orders using the following sequence:

1. Critical orders
2. Emergency orders
3. High-priority orders
4. Earlier required delivery date
5. Higher order value
6. Earlier order creation date

Java 8 concepts

- Comparator chaining
 - Custom priority mapping
 - `sorted()`
-

8.23 Find High-Value Orders

The user should define a minimum amount and retrieve all orders above that amount.

The system should support:

- Region-wise high-value orders
- Service-centre-wise high-value orders
- Monthly high-value orders

Java 8 concepts

- Predicate
 - Date filtering
 - Grouping
-

8.24 Identify Delayed Orders

An order is delayed when:

- It has not been delivered.
- Its required delivery date has passed.

The report should include:

- Delay in days
- Order priority
- Assigned warehouse
- Shipment status
- Responsible logistics provider

Java 8 concepts

- `LocalDate`
 - `ChronoUnit`
 - Filtering
 - Sorting
-

8.25 Identify Delayed Shipments

A shipment is delayed when:

- Actual delivery is missing, and
- Current date is after expected delivery date

or

- Actual delivery occurred after the expected delivery date

Java 8 concepts

- Optional
 - Date comparison
 - Predicate
-

8.26 Calculate Delivery Delay

Calculate the difference between:

- Expected delivery date
- Actual delivery date

Display the result in days.

Java 8 concepts

- Date and Time API
 - `ChronoUnit`
 - Function
-

8.27 Search Order by ID

Search an order using its unique ID.

The system should safely handle cases where the order is not available.

Java 8 concepts

- Optional
 - `findFirst()`
 - `orElse()`
 - `orElseThrow()`
 - `ifPresent()`
-

8.28 Search Shipment by Tracking Number

The system should find shipment details using a tracking number.

Comparison should be case-insensitive.

Java 8 concepts

- Stream filtering
 - Optional
 - Lambda expression
-

8.29 Group Orders by Status

Generate groups for:

- Created
- Approved
- Backordered
- Shipped

- Delivered
- Cancelled
- Returned

Java 8 concepts

- `Collectors.groupingBy()`
 - Enum-based grouping
-

8.30 Count Orders by Priority

Calculate the number of:

- Critical orders
- Emergency orders
- High-priority orders
- Normal orders
- Low-priority orders

Java 8 concepts

- `groupingBy()`
 - `counting()`
-

8.31 Partition Delivered and Undelivered Orders

Separate orders into:

- Delivered orders
- Undelivered orders

Java 8 concepts

- `partitioningBy()`
 - Predicate
-

8.32 Find the Highest-Value Order

Identify the order with the highest final payable amount.

The result should include:

- Order ID

- Service centre
- Order date
- Order value
- Assigned warehouse
- Order status

Java 8 concepts

- Comparator
 - `max()`
 - Optional
-

8.33 Find Top Ten Ordered Parts

Identify the ten parts with the highest total ordered quantity.

The report should combine quantities from all orders.

Java 8 concepts

- `flatMap()`
 - Grouping
 - Summing
 - Sorting
 - `limit()`
-

8.34 Identify High-Demand Parts

A part should be considered high demand when its monthly ordered quantity exceeds a configurable threshold.

Generate reports by:

- Part category
- Vehicle model
- Region
- Warehouse

Java 8 concepts

- Date filtering
- Flat mapping
- Grouping
- Aggregation

8.35 Identify Slow-Moving Parts

A part should be considered slow moving when:

- It has inventory available.
- It has had no order for a defined period.
- Its last issue date is older than the threshold.

Java 8 concepts

- Date and Time API
 - Predicate
 - `noneMatch()`
 - Grouping
-

8.36 Identify Dead Stock

Dead stock conditions:

- No movement for more than one year
- Part is obsolete or discontinued
- Inventory quantity is greater than zero

The system should calculate the financial value of dead stock.

Java 8 concepts

- Multiple predicates
 - Date comparison
 - Mapping
 - Reduction
-

8.37 Calculate Warehouse Inventory Value

Calculate inventory value as:

`Usable quantity × unit price`

Generate:

- Warehouse-wise inventory value
- Category-wise inventory value
- Region-wise inventory value

Java 8 concepts

- Grouping
- Summing
- Nested collectors

8.38 Find the Most Utilised Warehouse

Calculate warehouse utilisation based on:

`Current occupied capacity ÷ total storage capacity × 100`

Identify:

- Most utilised warehouse
- Least utilised warehouse
- Warehouses above 90% capacity

Java 8 concepts

- Function
- Comparator
- `max()`
- `min()`
- Optional

8.39 Supplier Performance Evaluation

Calculate a supplier performance score using:

- Quality rating
- On-time delivery percentage
- Average delivery time
- Rejected shipment count
- Order fulfilment rate

Example weightage:

Metric	Weight
Quality rating	30%
On-time delivery	30%
Fulfilment rate	20%

Metric	Weight
Delivery speed	10%
Rejection performance	10%

Java 8 concepts

- Custom functional interface
- Function composition
- Mapping
- Sorting

8.40 Classify Suppliers

Suppliers should be classified as:

Score	Category
90 and above	Strategic
75–89	Preferred
60–74	Approved
40–59	Improvement required
Below 40	High risk

Java 8 concepts

- Function
- Lambda expression
- Grouping

8.41 Find Best Supplier by Part Category

For each part category, identify the supplier with the highest performance score.

Java 8 concepts

- Grouping
- `maxBy()`
- Comparator
- Optional

8.42 Identify Supplier Contract Expiry

Generate reports for supplier contracts expiring within:

- 30 days
- 60 days
- 90 days

Java 8 concepts

- `LocalDate`
 - Date-range filtering
 - Sorting
-

8.43 Calculate Monthly Sales Revenue

Calculate delivered-order revenue month-wise.

Cancelled, returned and unpaid orders should be excluded according to business rules.

Java 8 concepts

- Date extraction
 - Grouping
 - Summing
 - Filtering
-

8.44 Calculate Region-Wise Revenue

Generate revenue summaries for:

- West
- North
- South
- East
- Central

Java 8 concepts

- `groupingBy()`
 - `summingDouble()`
-

8.45 Calculate Service-Centre Outstanding Amount

For each service centre, calculate:

Confirmed order amount – received payment – refund adjustments

Identify centres exceeding their credit limits.

Java 8 concepts

- Grouping
 - Reduction
 - Predicate
 - Mapping
-

8.46 Generate Order Summary Strings

Transform order objects into report rows containing:

- Order ID
- Service centre
- Priority
- Total amount
- Status
- Required delivery date

Java 8 concepts

- Function
 - `map()`
 - Method reference
-

8.47 Generate Unique Reference Numbers

Generate unique reference numbers for:

- Orders
- Shipments
- Payments
- Inventory transactions

Java 8 concepts

- Supplier

- Lambda expression
-

8.48 Validate Order Status Using Interface Static Method

Create a static interface-level validation operation to check whether an order-status transition is allowed.

Example:

- Created to Approved: allowed
- Delivered to Created: not allowed
- Cancelled to Shipped: not allowed

Java 8 concepts

- Static interface methods
 - Validation logic
-

8.49 Provide Default Shipping Policy

The shipment service should provide a default shipping policy.

The default method should explain:

- Standard delivery timeline
- Emergency shipment rules
- Partial shipment rules
- Delay escalation procedure

Individual logistics implementations may override the policy.

Java 8 concepts

- Default interface method
 - Interface evolution
-

8.50 Process Payment Asynchronously

Payment processing should execute without blocking order validation.

The asynchronous workflow should:

1. Validate payment details.
2. Process the payment.

3. Update payment status.
4. Update order status.
5. Generate transaction reference.
6. Send confirmation.

Java 8 concepts

- `CompletableFuture`
 - Asynchronous processing
 - Task chaining
-

8.51 Generate Reports Asynchronously

The application should generate the following reports simultaneously:

- Inventory report
- Order report
- Supplier performance report
- Shipment delay report
- Revenue report
- Low-stock report

Java 8 concepts

- Multiple `CompletableFuture` tasks
 - `allOf()`
 - Task combination
 - Result handling
-

8.52 Combine Warehouse Results

Different warehouses should independently calculate:

- Inventory value
- Low-stock parts
- Pending orders
- Delayed shipments

The system should combine all warehouse results into one national summary.

Java 8 concepts

- `CompletableFuture` combination
- Stream flattening
- Reduction

9. Recommended Layered Architecture

9.1 Model Layer

Suggested model entities:

- Part
- Supplier
- Warehouse
- Inventory
- ServiceCentre
- Order
- OrderItem
- Shipment
- Payment
- SupplierPerformance
- WarehouseReport

Responsibilities:

- Represent business data.
- Store entity properties.
- Maintain relationships between entities.
- Provide constructors and accessor methods.

9.2 DTO Layer

Suggested DTOs:

- OrderRequestDTO
- OrderSummaryDTO
- InventorySummaryDTO
- SupplierPerformanceDTO
- ShipmentStatusDTO
- PaymentRequestDTO
- DepartmentReportDTO

Responsibilities:

- Transfer data between layers.
 - Prevent direct exposure of model objects.
 - Provide report-specific representations.
 - Contain validated input data.
-

9.3 Repository Layer

Suggested repository contracts:

- PartRepository
- SupplierRepository
- WarehouseRepository
- InventoryRepository
- OrderRepository
- ShipmentRepository
- PaymentRepository

Responsibilities:

- Add records.
- Retrieve records.
- Search by ID.
- Update records.
- Delete or deactivate records.
- Return collections for Stream API processing.

Initially, repositories may use in-memory collections.

9.4 Service Layer

Suggested services:

- PartService
- InventoryService
- OrderService
- AllocationService
- PricingService
- SupplierService
- ShipmentService
- PaymentService
- ReportService
- NotificationService

Responsibilities:

- Apply business rules.
- Validate operations.
- Use Stream API.
- Calculate pricing.
- Allocate inventory.
- Generate reports.

- Execute asynchronous workflows.
-

9.5 Functional Interface Layer

Suggested custom functional interfaces:

DiscountCalculator

Accepts:

- Order amount
- Customer category
- Order priority

Returns:

- Discount amount

TaxCalculator

Accepts:

- Part category
- Taxable amount

Returns:

- Tax amount

ShippingCostCalculator

Accepts:

- Shipment weight
- Distance
- Priority

Returns:

- Shipping cost

SupplierScoreCalculator

Accepts:

- Supplier performance metrics

Returns:

- Supplier performance score

StockAllocationRule

Accepts:

- Inventory
- Requested quantity

Returns:

- Approved allocation
-

9.6 Utility Layer

Suggested utilities:

- ReferenceNumberGenerator
 - DateUtility
 - OrderValidator
 - AmountFormatter
 - InventoryCalculator
 - ReportFormatter
 - StatusTransitionValidator
-

9.7 Application Layer

Responsibilities:

- Load sample data.
 - Accept menu selections.
 - Call service operations.
 - Display reports.
 - Handle invalid input.
 - Coordinate complete business workflows.
-

10. Step-by-Step Implementation Guide

Step 1: Create the Project Structure

Create packages for:

- Model
 - DTO
 - Repository
 - Service
 - Functional interfaces
 - Utility
 - Exception
 - Application
-

Step 2: Define Enums

Create enums for:

- Part category
- Part status
- Order status
- Order priority
- Shipment status
- Payment status
- Supplier category
- Warehouse status
- Region

Enums should replace uncontrolled string values.

Step 3: Design Core Models

Create all business entities with suitable relationships.

Examples:

- An order contains multiple order items.
 - A warehouse contains multiple inventory records.
 - A shipment belongs to an order.
 - A payment belongs to an order.
 - A supplier supplies multiple part categories.
-

Step 4: Create DTOs

Create separate request and response structures.

Examples:

- Order creation request
 - Order pricing summary
 - Inventory availability response
 - Supplier performance summary
 - Shipment delay summary
-

Step 5: Prepare Sample Data

Create at least:

- 40 parts
- 10 suppliers
- 6 warehouses
- 100 inventory records
- 15 service centres
- 30 orders
- 70 order items
- 20 shipments
- 25 payments

Data should include:

- Active and inactive records
 - Available and unavailable stock
 - Expired and valid parts
 - Completed and delayed shipments
 - Paid and unpaid orders
 - Multiple order priorities
 - Different regions
-

Step 6: Create Repository Contracts

Define standard operations for each repository.

Include:

- Save
- Find all

- Find by ID
 - Find by business key
 - Update
 - Delete
 - Search by status
-

Step 7: Implement In-Memory Repositories

Use suitable collections to store data.

Ensure that:

- IDs remain unique.
 - Missing records are returned safely.
 - Duplicate records are rejected.
 - Repository operations do not directly contain business logic.
-

Step 8: Create Custom Functional Interfaces

Create functional interfaces for:

- Discount calculation
- Tax calculation
- Shipping calculation
- Supplier scoring
- Stock allocation

Each interface should contain one abstract method.

Step 9: Create Reusable Predicates

Create predicates for:

- Active parts
- Expired parts
- Critical parts
- Low-stock inventory
- Out-of-stock inventory
- Active suppliers
- High-risk suppliers
- Delayed orders
- Delayed shipments
- High-value orders

- Emergency orders

Combine predicates using logical composition.

Step 10: Create Reusable Functions

Create functions for:

- Converting parts into part summaries
 - Converting orders into order summaries
 - Calculating usable stock
 - Calculating order-item totals
 - Calculating delivery delays
 - Assigning supplier categories
 - Generating report lines
-

Step 11: Create Consumers

Create consumers for:

- Displaying order details
- Displaying inventory alerts
- Displaying shipment updates
- Displaying supplier scorecards
- Displaying report summaries

Use method references when possible.

Step 12: Create Suppliers

Use suppliers to generate:

- Order IDs
- Shipment IDs
- Payment IDs
- Inventory transaction IDs
- Tracking references

Ensure generated values are unique.

Step 13: Implement Part Management

Implement operations to:

- Add a part
 - Search a part
 - Update part status
 - Filter by category
 - Filter by vehicle model
 - Identify expired parts
 - Identify parts nearing expiry
-

Step 14: Implement Inventory Management

Implement:

- Stock addition
 - Stock reservation
 - Stock release
 - Stock issue
 - Damage recording
 - Usable-stock calculation
 - Low-stock alerts
 - Warehouse-wise inventory value
-

Step 15: Implement Order Validation

Validate:

- Service-centre status
- Order date
- Delivery date
- Order items
- Requested quantities
- Part status
- Duplicate parts
- Credit limit
- Emergency-order justification

Return all validation errors rather than stopping after the first error.

Step 16: Implement Warehouse Selection

For every order:

1. Check usable stock.
 2. Filter operational warehouses.
 3. Prefer the same region.
 4. Sort by delivery time.
 5. Select the best warehouse.
 6. Use multiple warehouses when necessary.
 7. Record unfulfilled quantities.
-

Step 17: Implement Pricing

Calculate:

1. Item totals
2. Gross order value
3. Discount
4. Tax
5. Shipping charges
6. Final payable amount

Generate a pricing breakdown.

Step 18: Implement Inventory Allocation

For each approved order:

- Reserve stock.
 - Prevent over-allocation.
 - Calculate backorder.
 - Create warehouse allocation records.
 - Update order status.
 - Generate allocation summary.
-

Step 19: Implement Partial Fulfilment

When stock is partially available:

- Approve available quantity.
- Store pending quantity.

- Search alternative warehouses.
 - Create multiple allocations.
 - Mark remaining quantity as backordered.
-

Step 20: Implement Order Prioritisation

Sort pending orders using:

- Priority
- Delivery date
- Order value
- Creation date

Generate a processing queue.

Step 21: Implement Shipment Management

Implement:

- Shipment creation
 - Tracking-number generation
 - Dispatch confirmation
 - Delivery confirmation
 - Delay calculation
 - Partial-delivery tracking
 - Shipment-status report
-

Step 22: Implement Payment Processing

Implement:

- Payment validation
 - Payment initiation
 - Payment confirmation
 - Payment failure handling
 - Refund calculation
 - Outstanding amount calculation
-

Step 23: Implement Supplier Analytics

Calculate:

- Performance score
 - Supplier category
 - Best supplier by category
 - High-risk suppliers
 - Contract expiry
 - Rejected delivery rate
 - On-time delivery performance
-

Step 24: Implement Order Analytics

Generate:

- Orders by status
 - Orders by priority
 - High-value orders
 - Delayed orders
 - Cancelled orders
 - Returned orders
 - Top service centres by order value
-

Step 25: Implement Inventory Analytics

Generate:

- Low-stock report
 - Out-of-stock report
 - Dead-stock report
 - Expiry report
 - Warehouse inventory value
 - High-demand parts
 - Slow-moving parts
-

Step 26: Implement Revenue Analytics

Calculate:

- Monthly revenue
- Region-wise revenue

- Warehouse-wise revenue
 - Service-centre-wise revenue
 - Category-wise revenue
 - Outstanding payments
-

Step 27: Implement Optional Handling

Use Optional for:

- Part search
- Supplier search
- Warehouse search
- Order search
- Shipment search
- Payment search
- Best warehouse selection
- Highest-value order
- Best supplier

Do not return null from search operations.

Step 28: Implement Default Interface Methods

Use default methods for:

- Standard shipping policy
- Default supplier-evaluation policy
- Default order-cancellation policy
- Default report heading

Allow implementations to override these policies.

Step 29: Implement Static Interface Methods

Use static interface methods for:

- Order-status transition validation
 - Salary or pricing validation
 - Payment amount validation
 - Shipment-date validation
 - Part-expiry validation
-

Step 30: Implement Asynchronous Workflows

Execute the following tasks asynchronously:

- Payment processing
- Stock reservation
- Shipment creation
- Notification generation
- Management report generation

Tasks should be chained according to dependency.

Step 31: Combine Asynchronous Results

Generate multiple warehouse and supplier reports simultaneously.

Combine the results into:

- National inventory dashboard
 - Consolidated supplier scorecard
 - National delayed-order report
 - Consolidated revenue summary
-

Step 32: Add Exception Handling

Create exceptions for:

- Part not found
 - Supplier not found
 - Warehouse not found
 - Order not found
 - Invalid order
 - Insufficient inventory
 - Credit limit exceeded
 - Invalid status transition
 - Payment failure
 - Shipment allocation failure
 - Duplicate reference number
-

Step 33: Generate Final Reports

The application should generate at least:

1. Active parts report
 2. Expired parts report
 3. Low-stock report
 4. Out-of-stock report
 5. Warehouse inventory-value report
 6. High-demand parts report
 7. Dead-stock report
 8. Pending orders report
 9. High-value orders report
 10. Delayed orders report
 11. Order fulfilment report
 12. Shipment delay report
 13. Supplier performance report
 14. Supplier contract-expiry report
 15. Revenue report
 16. Outstanding payment report
-

11. Complex Business Workflow

Complete Order Processing Flow

Phase 1: Order Creation

1. Service centre submits an order.
2. Order reference is generated.
3. Order items are validated.
4. Duplicate items are detected.
5. Required delivery date is validated.

Phase 2: Business Validation

1. Service-centre status is checked.
2. Credit limit is checked.
3. Part records are validated.
4. Expired or discontinued parts are rejected.
5. Emergency reason is validated.

Phase 3: Inventory Verification

1. Usable inventory is calculated.
2. Operational warehouses are identified.

3. Same-region warehouses are prioritised.
4. Inventory is allocated.
5. Partial and backordered quantities are calculated.

Phase 4: Pricing

1. Item totals are calculated.
2. Gross order amount is calculated.
3. Discount is applied.
4. Tax is calculated.
5. Shipping charges are calculated.
6. Final payable amount is generated.

Phase 5: Payment

1. Payment request is initiated asynchronously.
2. Payment status is updated.
3. Transaction reference is generated.
4. Payment failure is handled.

Phase 6: Fulfilment

1. Inventory is reserved.
2. Warehouse-wise shipment requests are created.
3. Tracking numbers are generated.
4. Order status is updated.
5. Service centre is notified.

Phase 7: Delivery

1. Shipments are tracked.
2. Delays are calculated.
3. Partial deliveries are recorded.
4. Final delivery is confirmed.
5. Reserved stock is converted to issued stock.
6. Order is closed.

12. Java 8 Feature Mapping

Java 8 feature	Case-study application
Lambda expressions	Pricing, validation, filtering and supplier scoring
Functional interfaces	Discount, tax, shipping and allocation rules
Predicate	Order, part, inventory and shipment validation

Java 8 feature	Case-study application
Function	Data transformation and calculations
Consumer	Report and notification display
Supplier	Unique reference-number generation
Method references	Data extraction and report display
Stream API	Collection processing
<code>filter()</code>	Selecting valid and matching records
<code>map()</code>	Transforming models into summaries
<code>flatMap()</code>	Processing order items across multiple orders
<code>sorted()</code>	Prioritising orders and ranking suppliers
<code>reduce()</code>	Calculating order and inventory totals
<code>groupingBy()</code>	Grouping by warehouse, region, status and category
<code>partitioningBy()</code>	Separating delivered and pending records
<code>counting()</code>	Counting orders and inventory records
<code>summingDouble()</code>	Revenue and inventory-value calculation
<code>averagingDouble()</code>	Supplier and order averages
<code>maxBy()</code>	Best supplier and highest-value order
<code>minBy()</code>	Earliest delivery or lowest-cost warehouse
<code>allMatch()</code>	Validating every order item
<code>anyMatch()</code>	Detecting critical or duplicate conditions
<code>noneMatch()</code>	Checking unwanted conditions
Optional	Safe missing-record handling
LocalDate	Order, expiry and delivery-date processing
ChronoUnit	Delay calculation
Default methods	Standard business policies
Static interface methods	Common validations
CompletableFuture	Payments, shipments and report generation

13. Advanced Assignment Tasks

1. Find the second-highest-value order.
 2. Find the highest-selling part in every region.
 3. Identify warehouses that cannot fulfil critical orders.
 4. Find suppliers supporting more than three part categories.
 5. Identify service centres exceeding their credit limits.
 6. Find parts ordered but never delivered.
 7. Calculate average shipment delay by logistics provider.
 8. Identify the logistics provider with the best delivery performance.
 9. Calculate warehouse-wise order-fulfilment percentage.
 10. Find orders fulfilled from more than one warehouse.
 11. Find parts supplied by multiple suppliers.
 12. Identify suppliers with declining performance.
 13. Find the top five service centres by revenue.
 14. Calculate revenue lost due to cancelled orders.
 15. Calculate the value of returned parts.
 16. Find orders containing at least one critical part.
 17. Identify orders in which every item was fully delivered.
 18. Find orders where no item was available.
 19. Generate a vehicle-model-wise demand report.
 20. Generate a part-category-wise profit report.
 21. Calculate supplier dependency for each critical part.
 22. Identify single-source critical parts.
 23. Generate a monthly stock-reorder plan.
 24. Generate emergency-order fulfilment statistics.
 25. Process every region's report using separate asynchronous tasks.
-

14. Testing Scenarios

Scenario 1: Empty Order

Expected result:

- Order should be rejected.
- A meaningful validation message should be generated.

Scenario 2: Duplicate Part

Expected result:

- Duplicate items should be identified.
- Quantities should either be merged or rejected.

Scenario 3: Insufficient Stock

Expected result:

- Available quantity should be allocated.
- Remaining quantity should be backordered.
- Stock should never become negative.

Scenario 4: Multiple Warehouses

Expected result:

- Order should be split warehouse-wise.
- Separate shipment requests should be created.

Scenario 5: Expired Part

Expected result:

- Expired part should not be allocated.
- An alternative part or error should be returned.

Scenario 6: Inactive Supplier

Expected result:

- Supplier should not be selected.
- Active alternatives should be checked.

Scenario 7: Payment Failure

Expected result:

- Order should not move to payment-confirmed status.
- Reserved inventory should be handled according to policy.

Scenario 8: Invalid Status Transition

Expected result:

- Transition should be rejected.
- Current order status should remain unchanged.

Scenario 9: Delayed Shipment

Expected result:

- Delay should be calculated.
- Shipment should appear in the delay report.

Scenario 10: Missing Actual Delivery Date

Expected result:

- Optional handling should prevent errors.
- Shipment should remain undelivered.

Scenario 11: Equal Supplier Scores

Expected result:

- Secondary ranking criteria should be applied.
- On-time delivery may be used as the tie-breaker.

Scenario 12: Asynchronous Task Failure

Expected result:

- Failure of one report should not crash all reports.
- A fallback message or partial result should be generated.

15. Final Deliverables

Learners should submit:

1. Project requirement document
2. Entity-design document
3. Package and layered-architecture diagram
4. Business-rule document
5. Stream-operation mapping
6. Order-processing workflow
7. Inventory-allocation workflow
8. Supplier-scoring design
9. Exception-handling design
10. Test-scenario document
11. Sample input data
12. Expected reports

- 13. Java implementation
 - 14. Final presentation or demonstration
-

16. Learning Outcomes

After completing this case study, learners should be able to:

- Design a complex domain model.
- Apply layered architecture.
- Use Java 8 functional interfaces.
- Create reusable predicates and functions.
- Process nested collections using streams.
- Use `flatMap()` for complex relationships.
- Perform multi-level grouping and aggregation.
- Build chained comparators.
- Handle missing data using `Optional`.
- Perform date-based business calculations.
- Define default and static interface methods.
- Design custom functional interfaces.
- Build asynchronous workflows using `CompletableFuture`.
- Combine results from multiple asynchronous operations.
- Apply Java 8 features to enterprise order, inventory and supply-chain workflows.